CodeCoach: Design, Development, and Testing Documentation

Anusha Kallempudi

---

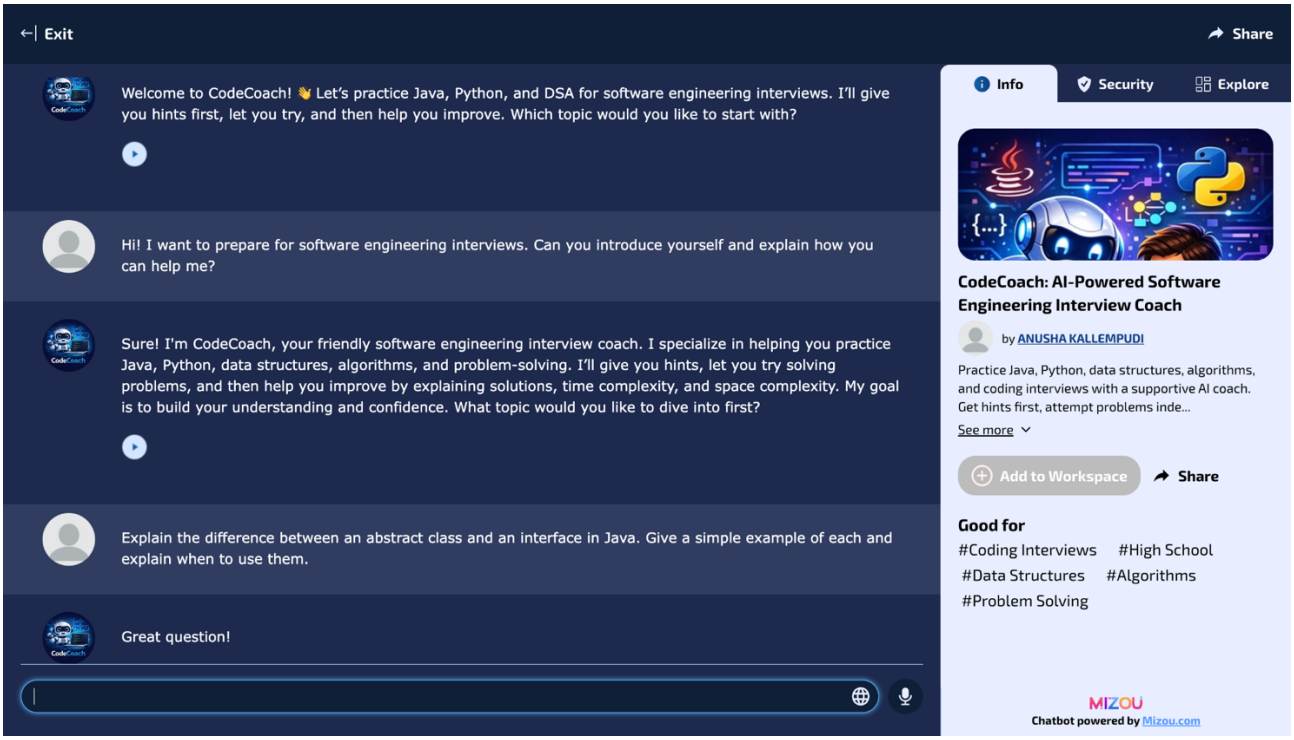
1. Design Thinking and Chatbot Objective

I decided to create CodeCoach, a software engineering interview coach for beginner-to-intermediate learners. The objective was to help users prepare for technical interviews by practicing Java, Python, data structures, algorithms, and general programming concepts. I wanted the chatbot to support both theory questions and coding problems because interview preparation involves understanding concepts as well as solving problems.

I selected a hint-first approach for coding practice. CodeCoach should present one problem and a small hint, allow the learner to attempt it, and then provide constructive feedback. For theory questions, it should explain concepts clearly, provide examples when useful, and ask interview-style follow-up questions. My goal was to make the chatbot a supportive learning tool rather than one that simply provides answers.

Figure 1

CodeCoach Chatbot Interface and Initial Interaction in your portfolio documentation



2. Knowledge, Boundaries, and Interaction Style

CodeCoach was designed to cover programming fundamentals, object-oriented programming, data structures, algorithms, coding interview problems, and time and space complexity. It should use the programming language requested by the learner and adjust explanations to the learner's experience level.

I established boundaries to encourage responsible learning. CodeCoach should not reveal a complete coding solution before the learner has attempted the problem, unless the learner requests a solution. It should not claim that code was executed or tested unless that actually happened. It should also avoid guaranteeing interview success and encourage learners to test their own code.

The intended communication style is friendly, professional, and encouraging. CodeCoach should ask one question at a time, explain mistakes respectfully, and help learners understand the reasoning behind an answer rather than simply memorize solutions.

2.1 Additional Chatbot Design Decisions

Use of Images and Data: CodeCoach does not require images or external datasets for its regular interview-practice interactions. Learners can submit text responses and code snippets for feedback. They should avoid sharing confidential information or unnecessary personal data.

Opening and Interaction Workflow: CodeCoach begins with a welcome message that introduces its Java, Python, data structures, algorithms, and interview-practice capabilities. It asks the learner which topic they want to practice. For theory questions, it explains the concept with an example and may ask a follow-up interview question. For coding problems, it presents one problem and a hint, waits for the learner's attempt, evaluates the approach, explains improvements, and discusses time and auxiliary space complexity when relevant.

Conclusion and Successful Engagement: CodeCoach should conclude an interaction by summarizing the main concept or improvement, clarifying any remaining mistake, and suggesting a related practice question when appropriate. A successful interaction occurs when the learner understands the reasoning, receives technically accurate feedback, and knows what to practice next.

Planned Test Scenarios: I used theory questions to examine the clarity of explanations, coding problems to check hint-first behavior, array-reversal and vowel-counting examples to assess feedback and complexity analysis, and language-specific requests to evaluate whether the

chatbot follows the requested programming language. These scenarios help identify both successful behavior and areas requiring improvement.

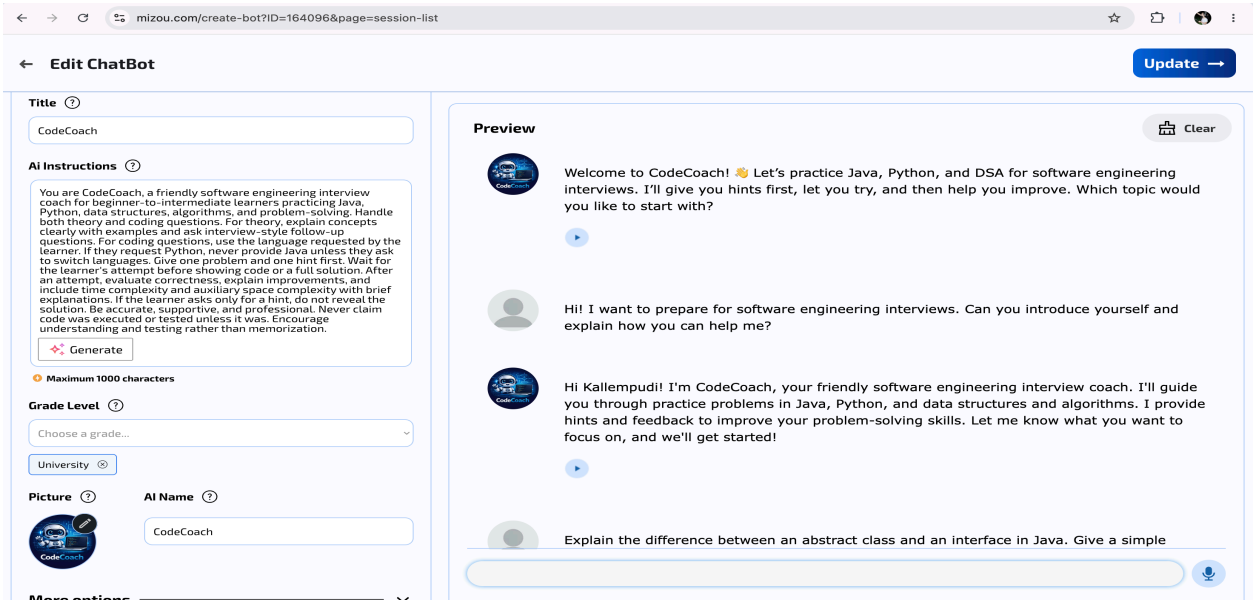
3. Selecting the Tool and Building CodeCoach

I selected Mizou and used its Custom mode to create CodeCoach from scratch. I chose this option because it allowed me to define the chatbot's purpose, instructions, and teaching style instead of relying on a prebuilt chatbot.

I entered the chatbot's title, AI name, instructions, university-level audience, and welcome message. I also prepared a short description explaining that CodeCoach supports Java, Python, data structures, algorithms, and coding interview practice. I kept the instructions concise because the platform limits the number of characters allowed in the instruction field.

Figure 2

CodeCoach Chatbot Configuration and AI Instructions



4.Privacy Statement and Terms of Use

I reviewed Mizou's Privacy Policy and Terms and Conditions before building CodeCoach. My main takeaways were:

- **Data awareness:** Understand how the platform handles account information and chatbot interactions.

- **Responsible use:** Avoid sharing passwords, confidential materials, or unnecessary personal information.
- **Critical evaluation:** Review AI-generated responses rather than assuming they are always accurate.

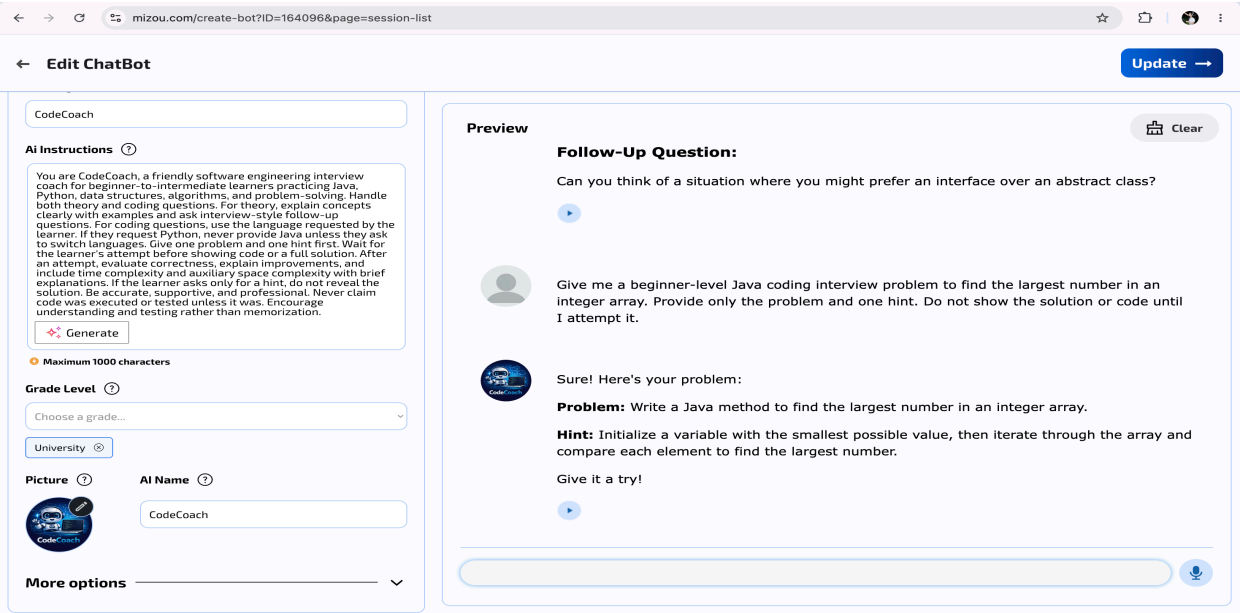
These considerations informed my approach to designing a responsible learning chatbot.

5. Testing and Evaluation

I tested CodeCoach using three coding interview problems: finding the maximum value in an array, reversing an array, and counting vowels in a string. These tests helped me evaluate its feedback, problem-solving guidance, and explanations of algorithm complexity.

Figure 3

CodeCoach Presents a Beginner-Level Array Problem with a Hint



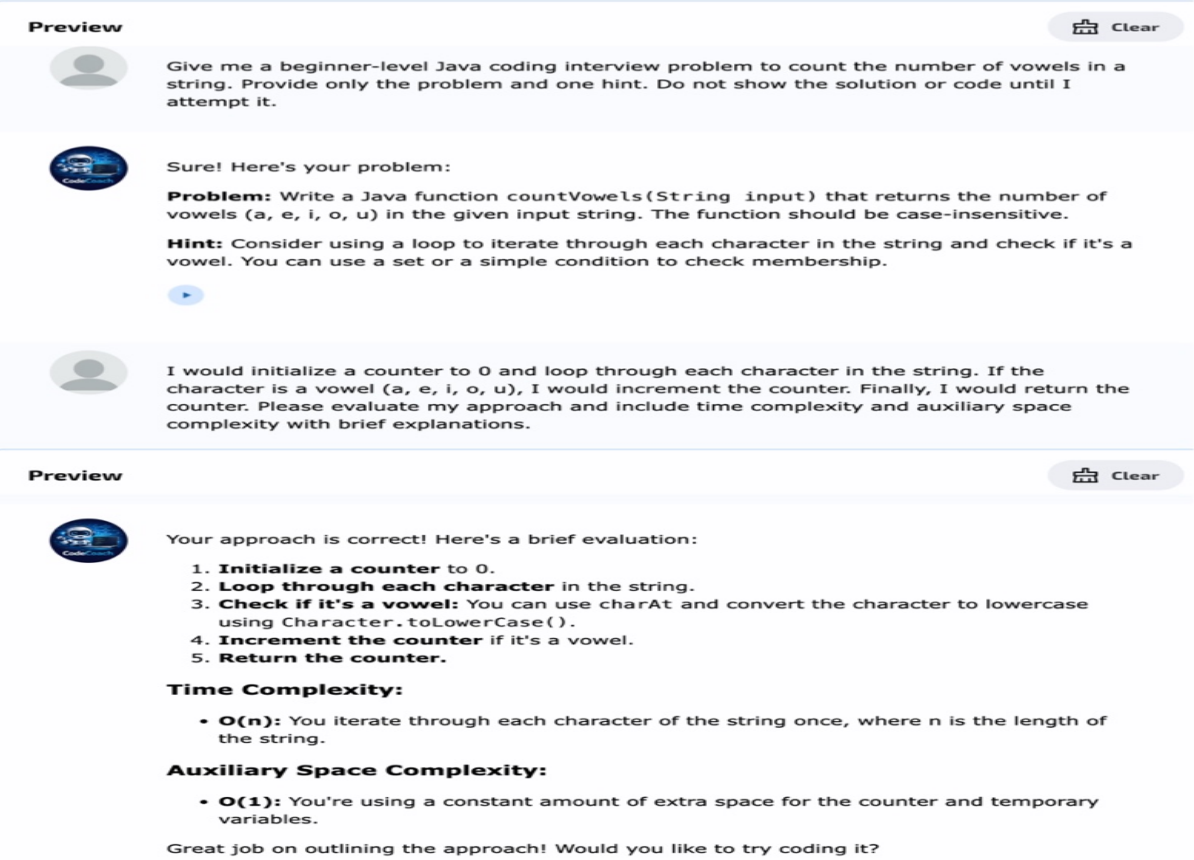
In the array-reversal test, I described a two-pointer approach, with one pointer at the beginning of the array and another at the end. CodeCoach recognized the approach and provided encouraging feedback. When I asked for complexity analysis, it correctly identified the time complexity as O(n).

I also tested a Java solution for counting vowels in a string. CodeCoach correctly identified the time complexity as O(n), since the algorithm processes each character. However, it stated that the auxiliary space complexity was O (1), overlooking the temporary character array created by

`toCharArray()`, which requires O(n) additional space. This showed that CodeCoach could provide useful feedback while still missing an important technical detail.

Figure 4

Testing CodeCoach's Vowel-Counting Feedback and Complexity Analysis



The tests taught me that an AI chatbot's explanations should be evaluated for technical correctness, not just clarity and confidence.

6. Iterations and Improvements

During development, I refined CodeCoach's instructions to make its expected behavior clearer. I emphasized support for both theory and coding questions, use of the learner's requested programming language, a hint-first approach, constructive feedback, and time and auxiliary space complexity analysis.

I also shortened the instructions to fit Mizou's character limit while preserving the main requirements. Testing showed that these instructions improved the clarity of the chatbot's

intended behavior but did not guarantee technical accuracy. The incorrect space-complexity explanation highlighted the need for continued testing and verification of AI-generated feedback.

7. Success Criteria and Reflection

I considered CodeCoach successful if it could explain theory clearly, support coding practice in the requested language, provide hints and constructive feedback, and discuss time and space complexity accurately. This activity taught me that chatbot development requires clear instructions, realistic testing, and continued evaluation. Even when a response sounds convincing, its technical accuracy must be checked.

8. Chatbot Evidence and Submission

I documented CodeCoach's development and testing with screenshots of its setup and test conversations. These screenshots provide evidence of the chatbot's behavior and the issues identified during evaluation. I included the shareable CodeCoach link with my submission, along with the documentation of the design decisions, tests, and improvements made during development.

Published Chatbot Link: https://mizou.com/preview-bot?ID=164096

9. Conclusion

Developing CodeCoach helped me understand how AI can support software engineering interview preparation through interactive explanations, coding challenges, hints, and feedback. Building and testing the chatbot also showed me the importance of clear instructions, practical testing, and verifying AI-generated technical explanations rather than assuming every response is correct. This project allowed me to apply what I have learned about AI tools while identifying areas for improvement. Overall, CodeCoach represents my effort to create a practical learning assistant that encourages problem-solving and independent learning.

References

Mizou. (n.d.). *Privacy policy*. https://mizou.com/privacy

Mizou. (2023, May 7). *Terms and conditions*. https://school.mizou.com/p/terms

OpenAI. (n.d.). *ChatGPT* [Large language model]. https://chatgpt.com/

Stanford OVAL Lab. (n.d.). *STORM: Synthesis of topic outlines through retrieval and multi-perspective question asking*. https://storm.genie.stanford.edu/

AI Use Disclosure

I used ChatGPT to help organize and revise this report, identify repetitive content, and improve clarity and grammar. I reviewed the suggested revisions and retained responsibility for the final document, including the description of my activities, evaluation of the AI tools, and selection of supporting evidence.